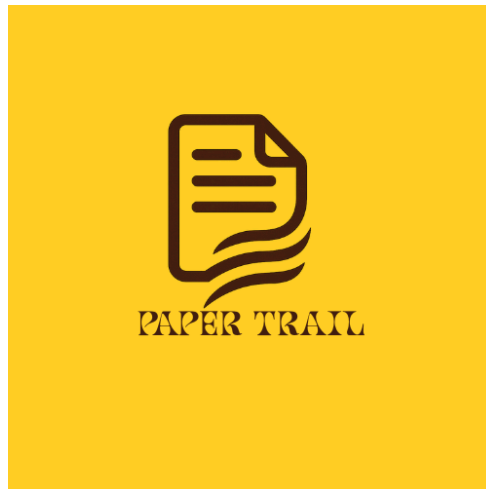




Paper Trail
System Design Document

Dr. Zihan Wu

Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, Vasu Patel

11/17/2025

V1.0

Table of Contents

1	Introduction.....	3
1.1	Purpose of This Document.....	3
1.2	References.....	3
2	System Architecture.....	4
2.1	Architectural Design.....	4
2.2	Decomposition Description.....	6
3	Persistent Data Design.....	7
3.1	Database Descriptions (if you use a database).....	7
3.2	File Descriptions.....	9
4	Requirements Matrix.....	9
	Appendix A – Agreement Between Customer and Contractor.....	10
	Appendix B – Team Review Sign-off.....	11
	Appendix C – Document Contributions.....	12

1 Introduction

This is a capstone project commissioned by Dr. Zihan Wu. This project will fulfill the capstone requirement for a Computer Science Bachelor's degree for Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, and Vasu Patel. As a researcher, the client has expressed frustration with the difficulty that comes with maintaining a single “main” document where progress, notes, and references are collected. They noticed that Google Docs doesn't natively support project management functions. When they tried third-party applications like Notion and Asana, they found them very convoluted and unintuitive. This project aims to offer an intuitive plugin that works directly within Google Docs while writing. The client proposed a tagging system in which any paragraph that follows a special character (“#”, “@”, etc.) is treated as a note until a page break or another tag. Thus, differentiating notes from actual documentation. The client also requested multiple types of tags (type of note, project it's under, date, etc.) to allow for more targeted searching functionalities. Also, they requested a stretch goal: the ability to do quick commands with backslashes while typing in the document to bring up to-do tables and such on the fly.

1.1 Purpose of This Document

The purpose of this document is to outline a high-level overview of the system architecture and the low-level design specifications. This document will clarify all technologies, tools, databases, etc. that are to be used for the development of the PaperTrail product and its requirements. This document will also go into detail about how said technologies will be structured and implemented, and what their purpose is.

1.2 References

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Requirements Specification*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail User Interface Design Document*

2 System Architecture

This section will demonstrate the functions and architecture of the plugin. How it will be utilized by the user, how it will react to the established systems provided by Google, and how it will avoid infringing on any cybersecurity policies. By the end of this section, readers should have a clear understanding of the scope and extent of this project.

2.1 Architectural Design

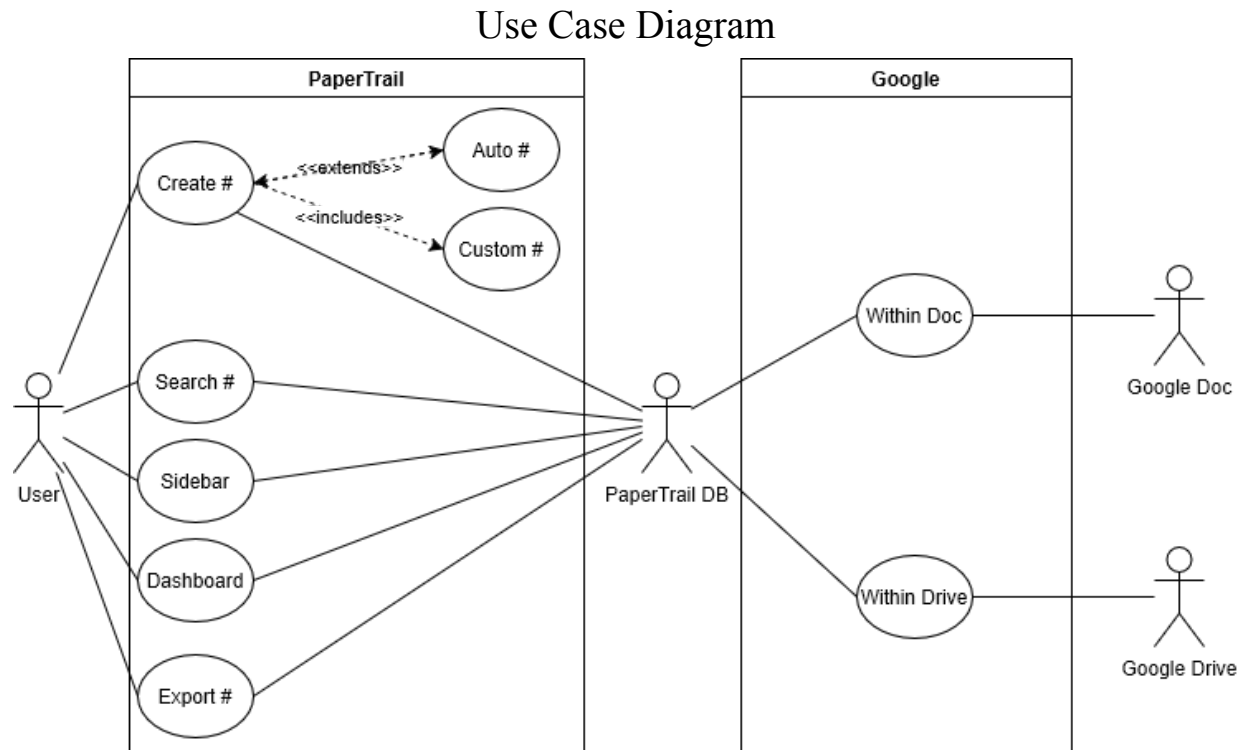


Figure 1: A use case diagram that shows the functions that the user will be able to access, and the systems the software will delegate for the user.

This use case diagram illustrates how the users will interact with the software, and how the software will react to Google's systems. The user will be able to interact with the front-end functions such as creating tags, searching tags, sidebar functionality, dashboard functionality, and a function to export tags. Each front-end component is linked to corresponding backend processes or systems. For example, the sidebar integrates with Google Docs, while the dashboard connects to Google Drive. Each function directly corresponds to the PaperTrail database.

The software will partially rely on Google's API for Google Docs and Drive. The plugin will host its own backend that is local to the machine. This will be done by using React to be a frontend system that will shape the way Google Docs looks by providing HTML and CSS scripts. On the backend, additional functionality for Google Docs and Drive will be implemented via JavaScript and GoogleScript languages.

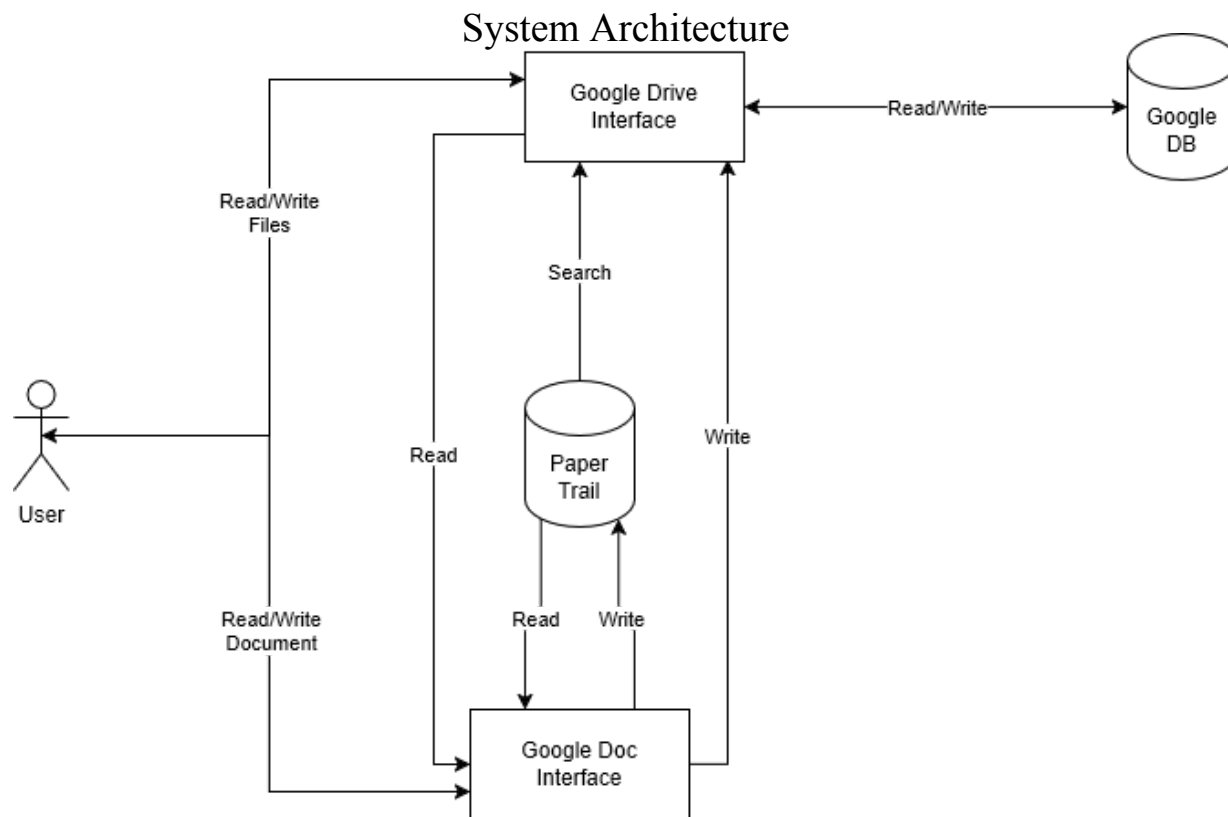


Figure 2: A system architecture diagram that shows how the user interacts with Google's systems will affect the software.

The system architecture diagram above illustrates that PaperTrail will have access to read and write privileges to the user's documents, and the search functionality provided by PaperTrail in the user's personal Google Drive. This will make it so that there are no infringements with Google's private database when Paper Trail makes updates to users' documents; The only instance where an update is pushed to Google's servers is through Google Drive, which Google Docs does automatically whenever changes are detected.

The paper trail system is meant to be an assistant software that helps the user write up documents, while easily having access to information across the documents they have in their Google Drive. Which is why PaperTrail doesn't have direct functionality with the user, instead working off the information provided by the user's documents found in their own Google Doc and Drive. This is done by letting PaperTrail read and make modifications that the user provides, and can search throughout their Google Drive to find tags that have been requested by the user in the document they are writing.

2.2 Decomposition Description

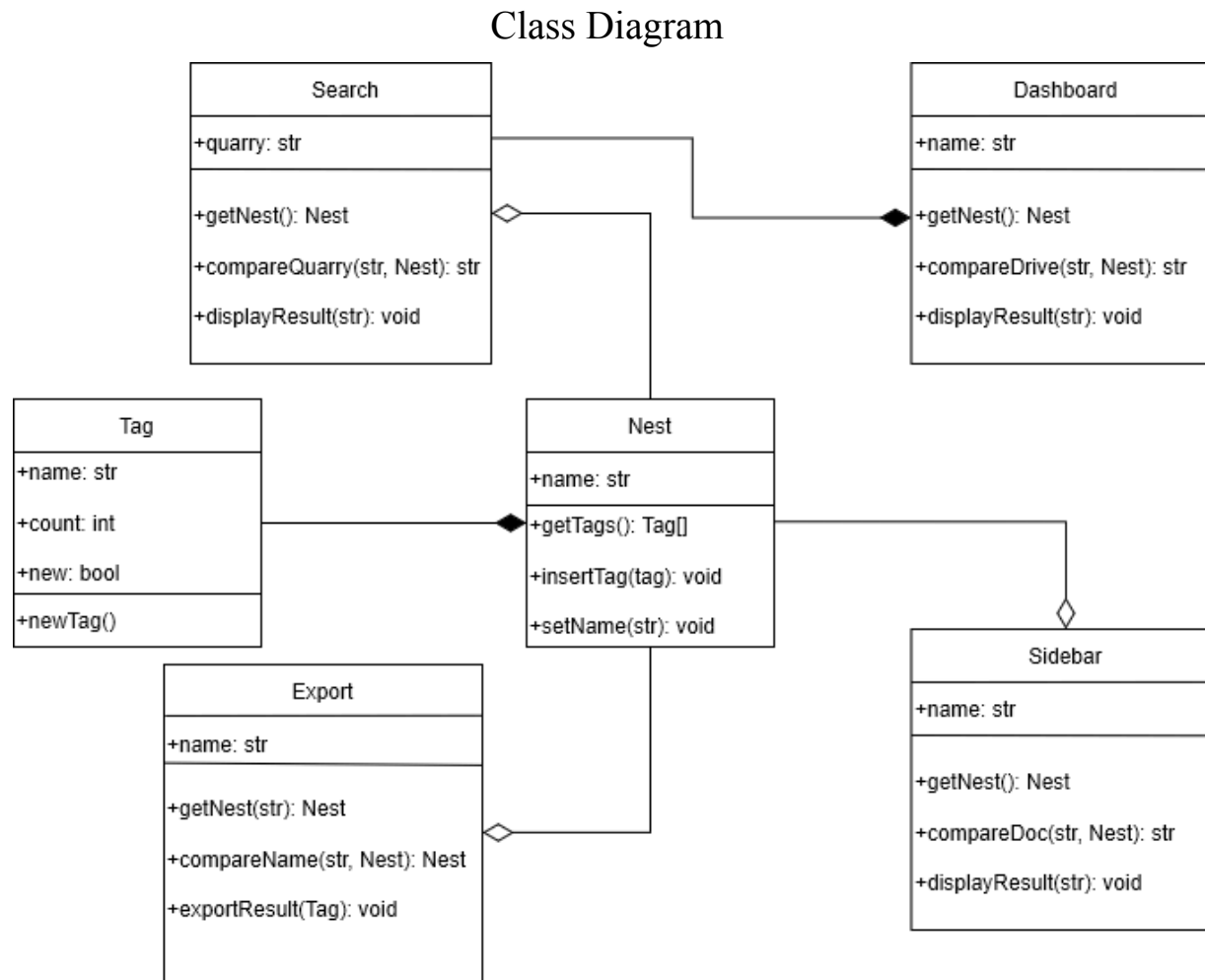


Figure 3: A class diagram that shows how each function will operate, and how they connect to the other functions when instantiated.

The Class Diagram above illustrates the classes that will be used by the software. Each Tag object will be categorized into a Nest object that will house an array of Tags that have been associated together by the user. Later will be accessed by the other Classes like Search, Export, Sidebar, and Dashboard. The Export class will take the requested nest of tags and export them onto a text file with their tagged notes, allowing the user to have them for personal use. The Search Class will take the queried input and compare it against the nest of tags objects, returning any that match the query, and displaying them to the user. Lastly, the Dashboard class found in the user's Google Drive, and the Sidebar class will be accessed while the user is writing in Google Docs.

3 Persistent Data Design

This section describes the permanent storage structure used by PaperTrail. The system uses a MySQL database to maintain records of documents, supported tags, and the locations where those tags appear within documents. This persistent data enables search functionality and autocomplete suggestions. The following section details each database table, its purpose, and the fields necessary for its implementation.

3.1 Database Descriptions (if you use a database)

Database Diagram

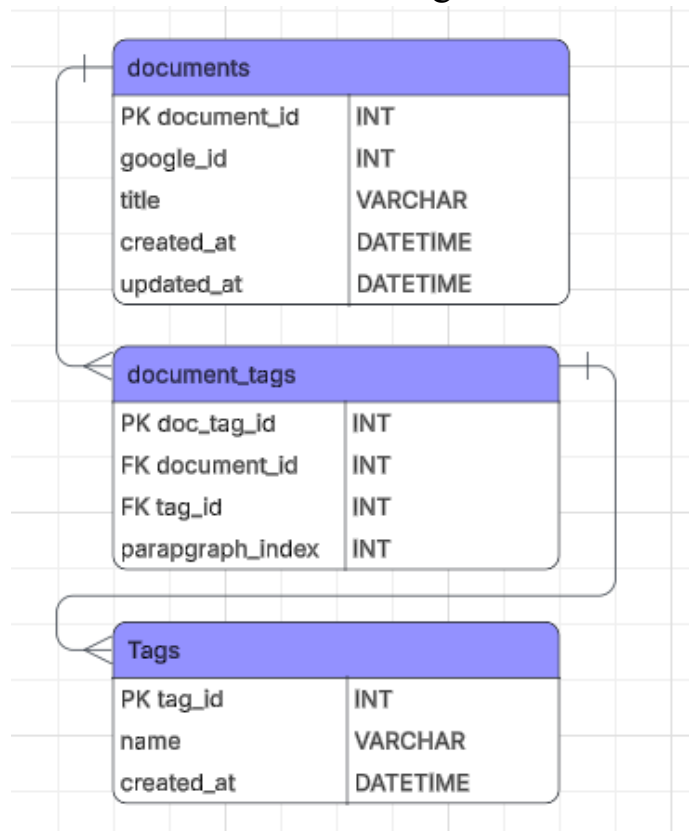


Figure 4: The database diagram shows how each table is related.

documents Table:

Stores basic information about every Google Doc that the plugin tracks.

Field Name	Purpose
document_id (Primary Key)	Internal unique ID for each document in the database.
google_id	The Google Docs unique identifier extracted from the document's URL. Used to reconnect to the document later.
title	The readable name of the document at the time it was first recorded.
created_at	Timestamp of when the system first indexed the document.
updated_at	The last time the plugin re-indexed the document or updated its metadata

document_tags Table:

This links specific tags to specific locations inside a document.

Field Name	Purpose
Doc_tag_id (Primary Key)	Unique ID for each tag assignment. One row = one tagged paragraph. Each tag has one string of characters.
Document_id (Foreign Key)	Which document the tag appears in.
Tag_id (Foreign Key)	Which tag was used
paragraph_index	The paragraph number/line number where the tag is located inside the Google Doc.

Tags Table:

Stores the tags to be used by the system.

Field Name	Purpose
Tag_id PK	Internal unique ID for each tag
name	The text of the tag (example: #Note).
created_at	Timestamp of when the tag was added to the system

3.2 File Descriptions

No flat file storage is used. All persistent data is stored directly in MySQL. Google Docs content is accessed dynamically through the Google Docs API and is not saved locally.

4 Requirements Matrix

This section provides a matrix linking the functional requirements from the PaperTrail System Requirements Specification to their corresponding system components. Each requirement has a unique identifier that is mapped to the interfaces or modules that satisfy it.

The table below shows which architectural or design components (frontend modules, backend services, etc.) fulfill each requirement.

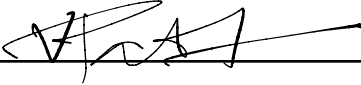
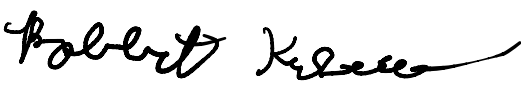
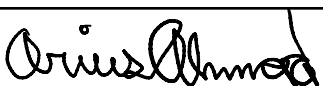
SRS ID	Name	Component/Module
1	Create and Categorize Nested Hashtags	Frontend - React UI Backend - Google Script, Tag Index System
2	Search within Document by Tag	Frontend (Search UI Component) Backend - (Search Handler), Google Docs API
3	Create Custom Hashtag Template	Template Manager (UI Modal), Tag Database
4	Autocomplete Hashtag	Frontend (Autocomplete Module) Backend Tag Priority System
5	Search/Filter by Documents Containing Tags	Backend (Google Drive API) Frontend Search Dashboard
6	Sidebar Suggestions in Google Docs	Frontend (Sidebar Component) Backend (Suggestion Engine)
7	Google Drive Dashboard View	Frontend - Dashboard UI within Drive Backend - Drive API
8	Export Tagged Notes	Backend (Export Handler), Drive API File Generator (PDF etc.)

Appendix A – Agreement Between Customer and Contractor

This document represents a formal agreement between the PaperTrail Development Team and Dr. Zihan Wu. The agreement confirms that both parties understand and accept the scope, objectives, and deliverables of the *PaperTrail Project*, which aims to create a document tracking and workflow automation system integrated with Google Docs and Google Drive. Both parties agree to collaborate throughout the project lifecycle, with the development team responsible for implementing the system according to the specifications outlined in this Software Design Document, and the customer providing timely feedback and testing support.

In the event of future changes to this document, all revisions must be proposed in writing by either party. Any modification will require a review meeting between the customer and the contractor, and mutual written approval before updates are considered valid. Updated versions of the SDD will be labeled clearly with version numbers and dates to ensure traceability.

Signatures

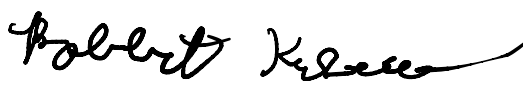
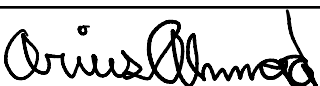
Name (Typed)	Signature	Date
Dr. Zihan Wu (Client)	<i>Zihan Wu</i>	Nov 17, 2025
Vasu Patel		Nov 17, 2025
Ben Yandell		Nov 17, 2025
Robert Kulow		Nov 17, 2025
Arius Ahmad		Nov 17, 2025
Anthony Veilleux		Nov 17, 2025

Client Comments (if applicable)	
---------------------------------	--

Appendix B – Team Review Sign-off

All members of the PaperTrail Development Team have thoroughly reviewed this Software Design Document and agree that it accurately represents the project's goals, scope, and requirements. Each team member confirms that the content and format of this document meet the expectations of both the team and the customer. While minor editorial or stylistic differences may exist, there are no major points of disagreement among the team regarding the material presented herein.

Team Member Signatures

Name (Typed)	Signature	Date
Vasu Patel		Nov 17, 2025
Ben Yandell		Nov 17, 2025
Robert Kulow		Nov 17, 2025
Arius Ahmad		Nov 17, 2025
Anthony Veilleux		Nov 17, 2025

Comments	
----------	--

Appendix C – Document Contributions

The following table identifies each team member's contributions to the creation of this Software Design Document. Each member has contributed to writing, editing, and reviewing the document. The percentages represent an estimate of each member's contribution to the total effort.

Team Member	Contributions	Percentage of Work
Vasu Patel	Did review of all sections and edited the document based on the peer comments	20%
Ben Yandell	Added Requirements Matrix (Section 4)	20%
Robert Kulow	Major portions on section 2.	20%
Arius Ahmad	Completed section 1.	20%
Anthony Veilleux	Section 3	20%

